Automated SCADA Dashboard Generation from Firmware Configuration Headers

Department of Electrical Engineering and Information Technology

Chair of Control Systems

RPTU University Kaiserslautern-Landau

Submitted by:

Sara Bavi
Student ID: 427144

February 23, 2026

Contents

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Project Overview . . . 2

2 Concept and Workflow **3**
2.1 System Architecture . . . 3
2.2 Data Flow Logic and Determinism . . . 4
2.2.1 Type Safety and Encoding . . . 4
2.2.2 Reproducibility and Drift Elimination . . . 4
2.3 Data Transformation Example: Temperature Signal . . . 4

3 Implementation Details **6**
3.1 Parsing the Configuration Header . . . 6
3.2 Generating the Node-RED Flow . . . 7
3.2.1 Runtime Arguments . . . 8
3.3 Dashboard Layout and Deployment . . . 8

4 System Verification and Validation **9**
4.1 Pipeline Verification . . . 9
4.2 System Validation via PLECS Simulation . . . 10
4.2.1 End-to-End Functional Testing . . . 11

5 Conclusion and Future Work **12**
5.1 Conclusion . . . 12
5.2 Future Work . . . 12

Abstract

This report presents a fully automated, configuration-driven toolchain designed to generate Supervisory Control and Data Acquisition (SCADA) dashboards directly from embedded firmware definitions. Traditionally, synchronizing field-device firmware with Human-Machine Interfaces (HMIs) requires manual engineering, which is prone to integration errors and scalability limitations. To address this, a modular Python-based pipeline was developed to parse C-language configuration headers, extract physical parameters and metadata, and dynamically synthesize deployable Node-RED JSON flow files. The generated dashboard accurately reflects the firmware's data structures, utilizing dynamically routed JavaScript nodes for binary payload decoding and MQTT communication. The system's robustness was successfully verified and validated through a digital twin approach using a PLECS simulation of a Boost converter. The results demonstrate that this framework effectively eliminates configuration drift, ensures structural consistency, and significantly reduces development overhead for power electronics monitoring and control systems.

Chapter 1

Introduction

This report documents the development of a modular Automated SCADA (Supervisory Control and Data Acquisition) generation framework. While the system's validation is primarily demonstrated using a Boost converter in this project, the underlying architecture is designed to be device-agnostic. It is equally applicable to a wide range of power electronics equipment, including Supercapacitors, Battery Loads, and Active Rectifiers.

The primary objective of this project is to streamline the creation of visualization dashboards by deriving them directly from the embedded firmware configuration. By decoupling the visualization logic from specific hardware definitions, the tool ensures scalability and rapid adaptability across diverse hardware platforms.

1.1 Motivation

In the development of power electronics, firmware configurations evolve rapidly. Typically, developers maintain a C-header file (e.g., `boostInfConfig.h`) that defines data structures and communication parameters. However, monitoring these parameters often requires a separate Human-Machine Interface (HMI) that must be manually updated to match the firmware.

This manual synchronization presents several challenges:

- **Scalability Issues:** As the number of supported devices (Boost, SuperCap, Battery Load) grows, maintaining separate dashboards for each becomes unmanageable.
- **Human Error:** Manual updates often lead to mismatched data types or incorrect MQTT topic assignments.
- **Development Overhead:** Significant time is lost re-configuring dashboards rather than testing the control logic.

The motivation for this project is to eliminate this overhead by treating the firmware header file as the "single source of truth." This allows for the automatic generation

of dashboards for any supported equipment type without manual intervention.

1.2 Project Overview

The solution is a Python-based toolchain that converts C-language configuration files into deployable Node-RED flows. As detailed in the project repository, the system supports dynamic configuration for various device types.

The architecture consists of three main stages:

1. **Configuration Parsing:** The script `parse_config_header.py` analyzes the specific header file (e.g., `boostInfConfig.h` for Boost converters, or equivalent headers for other devices). It extracts metadata and exports it to an intermediate JSON format.
2. **Flow Generation:** The `generate_flow.py` script constructs a complete Node-RED flow. It dynamically assigns MQTT topics and UI widgets based on the device type defined in the configuration.
3. **Deployment:** The generated JSON file is imported into Node-RED, instantly creating a dashboard tailored to the specific equipment (e.g., Boost, SuperCap) defined in the source header.

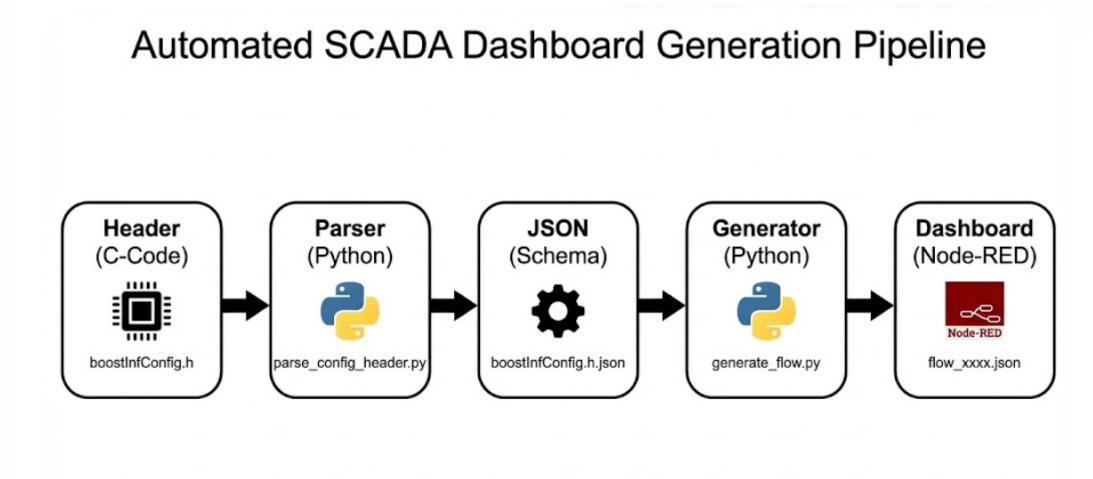

Figure 1.1: Overview of the Automated SCADA generation

Chapter 2

Concept and Workflow

This chapter outlines the conceptual framework of the automated dashboard generation system. The core principle is to utilize the embedded firmware's configuration header as the authoritative single source of truth for the SCADA interface. By implementing a configuration-driven automation framework, the system ensures structural consistency between the embedded device firmware and the supervisory visualization layers.

2.1 System Architecture

The system architecture eliminates manual dashboard configuration by establishing a fully automated transformation from embedded firmware definitions to deployable SCADA visualization flows. This is achieved through a unidirectional data pipeline consisting of three sequential stages:

1. **Configuration Abstraction Layer (Source & Parsing):**`boostInfConfig.h` uses Regular Expressions to transform C-structs and enums into a structured JSON Schema. This ensures that any change in the firmware (adding a new fault, state, Signals,..) automatically updates the SCADA definition without manual coding.
   - **Static Extraction:** The script `parse_config_header.py` analyzes these definitions along with metadata such as engineering units, plot groups, Bit-mapped Fault.
   - **Canonical Model:** The parser converts this into a normalized JSON representation (`boostInfConfig.h.json`), which functions as a canonical intermediate representation.
2. **SCADA Flow Synthesis Layer (Processing & Generation):** This stage is executed by `generate_flow.py`, which consumes the JSON configuration to synthesize a complete Node-RED flow definition.
   - **Dynamic Construction:** The generator creates MQTT nodes, Function nodes and UI widgets based on the configuration content.

- **Visualization Logic:** It automatically constructs dynamic widgets and trend tools based on the extracted definitions.

3. **Supervisory Deployment Layer (Presentation & Runtime):** In the final stage, the generated flow file (`flow_<timestamp>.json`) is imported into the Node-RED runtime environment.
   - **Communication:** Upon deployment, the system establishes MQTT communication with the specified broker.
   - **Control & Rendering:** The system renders the HMI dashboard according to the equipment type and enables control command publishing.

2.2 Data Flow Logic and Determinism

The automation relies on strict, deterministic parsing logic. The header file serves as a hierarchical map; for example, a C struct containing voltage parameters translates directly to specific MQTT topic structures.

2.2.1 Type Safety and Encoding

The generation process handles the translation of C-types into JavaScript logic. The generator constructs specific function nodes for signal decoding and command encoding to ensure data integrity between the binary firmware and the JSON-based Node-RED environment.

2.2.2 Reproducibility and Drift Elimination

The process is entirely configuration-driven. Any modification in the original header file requires only the re-execution of the two Python scripts to produce an updated flow.

2.3 Data Transformation Example: Temperature Signal

To clearly illustrate how the automated pipeline functions, we can trace the path of a single variable from the embedded firmware up to the visual dashboard. The workflow follows the sequence: **Header** → **Parser** → **JSON** → **Generator** → **Dashboard**.

Consider the temperature signal, `th`, defined in `boostInfConfig.h`. The firmware developer adds metadata using standard style comments. These comments dictate how the signal should be visualized, including its human-readable label, physical unit, and which chart group it belongs to.

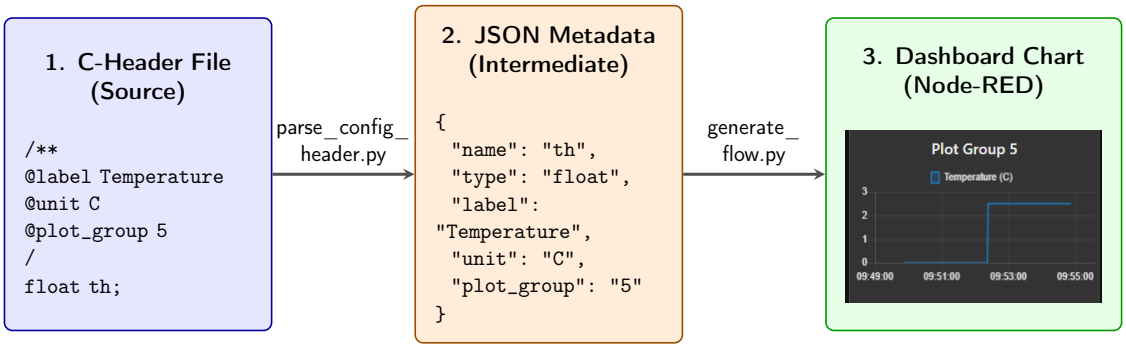

Figure 2.1: Example of the data flow for a single variable (`th`) from its initial C-header definition to the final Node-RED dashboard visualization.

Chapter 3

Implementation Details

This chapter details the software implementation of the generation pipeline, highlighting the deterministic transformation of embedded C-code into a deployable Node-RED dashboard.

3.1 Parsing the Configuration Header

The first stage of the pipeline is handled by `parse_config_header.py`. This script utilizes regular expressions (regex) to parse the C-style struct definitions, enums, and macros found in `boostInfConfig.h`. It dynamically identifies and extracts:

- Telemetry signal definitions
- Hardware parameters
- Control parameters
- Bit-mapped fault flags
- State machine enumerations
- Commands
- The logical framework for real-time device status and MQTT communication structures

To ensure precise metadata extraction without disrupting the C-code parsing, the following Python logic is utilized within `parse_config_header.py`:

```python
def extract_doc_block(before_text):
    comment_match = re.search(r"/\*\*([\s\S]*?)\*/\s*$",
        before_text, re.DOTALL)
    if not comment_match:
        return {"label": "", "unit": "", "plot_group": ""}

    comment = comment_match.group(1)
    label_match = re.search(r"@label\s+([^\n@]*)", comment)
    unit_match = re.search(r"@unit\s+([^\n@]*)", comment)
    plot_group_match = re.search(r"@plot_group\s+([^\n@]*)",
        comment)

    return {
        "label": label_match.group(1).strip() if label_match else "
            ",
        "unit": unit_match.group(1).strip() if unit_match else "",
        "plot_group": plot_group_match.group(1).strip() if
            plot_group_match else ""
    }
```

Listing 3.1: Python Regex helper function for extracting metadata annotations from the C-header.

3.2 Generating the Node-RED Flow

The `generate_flow.py` script ingests the intermediate JSON and transforms it into a Node-RED compatible format. It programmatically constructs specific Node-RED UI nodes (e.g., `ui_tab`, `ui_group`, `ui_chart`, `ui_template`) and interconnects them with MQTT broker nodes.

To demonstrate this data handling logic, the following JavaScript snippet is dynamically injected by `generate_flow.py` into the Node-RED function nodes. This securely parses incoming binary MQTT payloads into 32-bit floating-point values for the dashboard charts:

```javascript
function bytesToFloat32(buf, offset) {
    const view = new DataView(buf.buffer, buf.byteOffset, buf.
        byteLength);
    return view.getFloat32(offset, true);
}

let buf = msg.payload;
if (!Buffer.isBuffer(buf)) return null;

// The Python generator dynamically injects the JSON array of
    parameters below:
const params = ["v_in", "v_out", "i_in"];
// ... extraction logic continues
```

Listing 3.2: Dynamic JavaScript injection for decoding Float32 MQTT binary payloads within Node-RED.

3.2.1 Runtime Arguments

In addition to the configuration JSON, the generator script accepts runtime parameters via the command line interface:

- `--broker <BROKER_IP>` (Default: `mosquitto`)
- `--prefix <TOPIC_PREFIX>` (Default: `boost_inf_1`)

These parameters define the MQTT broker endpoint and the topic namespace prefix for the specific equipment instance. This design allows the exact same firmware configuration to be deployed across multiple physical devices without modification, simply by altering the namespace prefix at runtime.

3.3 Dashboard Layout and Deployment

The generated layout is organized into a cohesive UI tab divided into functional groups: Status, State, Commands, Signals, Faults, and Parameter panels (Hardware and Control).

The final output of the script is a time-stamped JSON file (e.g., `flow_<YYYYMMDD_HHMMSS>.json`). Importing this file directly into the Node-RED environment instantly deploys the updated dashboard, establishing immediate MQTT subscriptions to the defined broker and rendering the interface based strictly on the firmware's definitions.

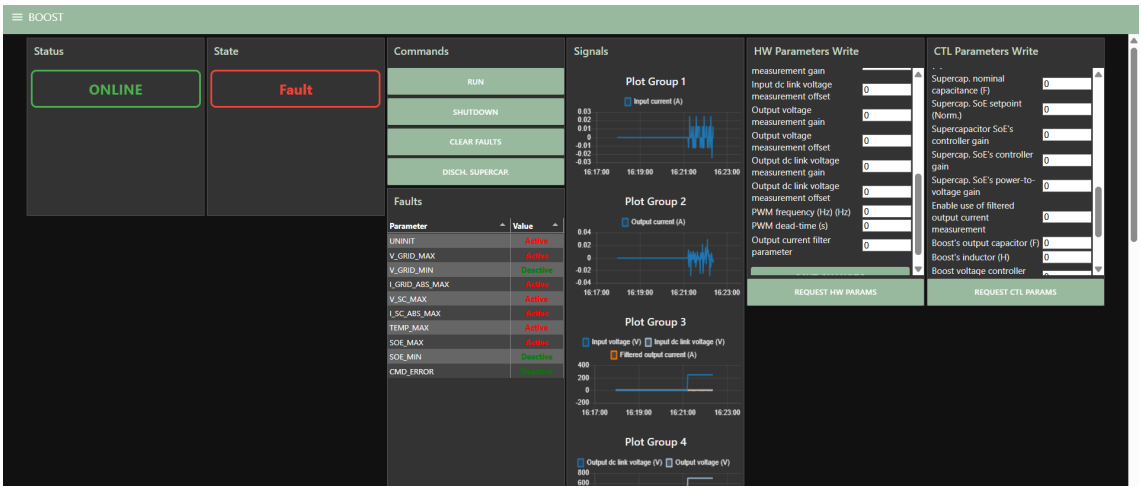

Figure 3.1: Final dynamic Node-RED dashboard generated from the C-header configuration.

Chapter 4

System Verification and Validation

To ensure the reliability and functional correctness of the automated SCADA generation toolchain, a comprehensive testing strategy was implemented. This chapter outlines the verification of the software pipeline and the dynamic validation of the generated Node-RED dashboard using a simulated environment.

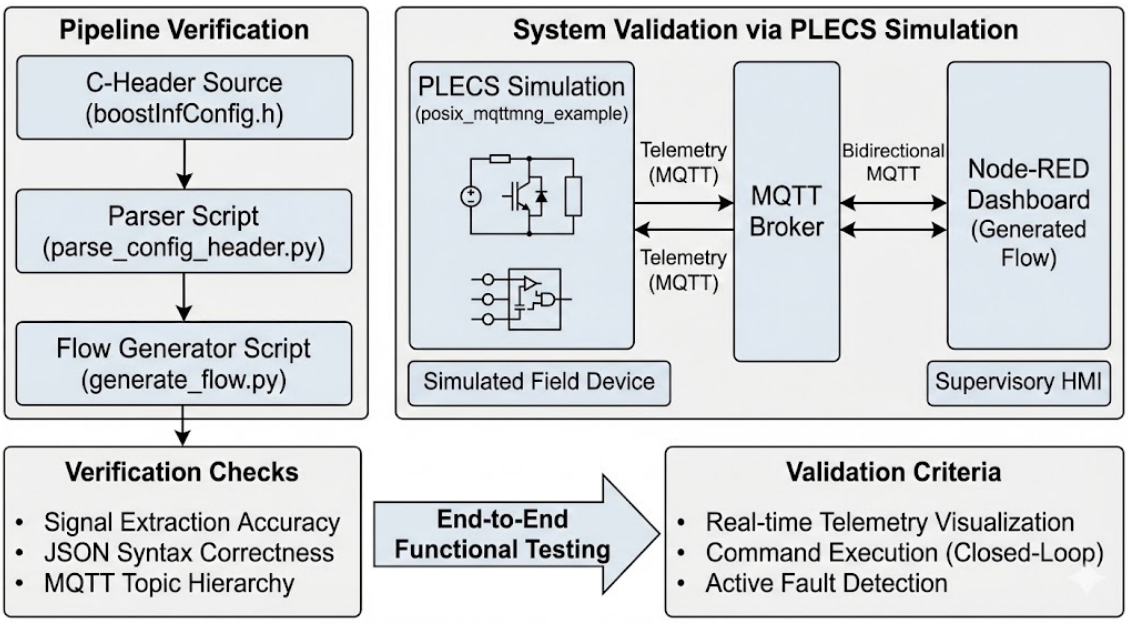

Figure 4.1: Overview of the verification and validation methodology, illustrating the pipeline verification checks and the closed-loop system validation via PLECS simulation.

4.1 Pipeline Verification

Verification ensures that the translation from the C-header source to the Node-RED flow strictly adheres to the defined system specifications. This phase of testing focused on the deterministic behavior of the Python scripts:

- **Parser Accuracy:** Verifying that `parse_config_header.py` correctly extracted all signals, data types, and metadata without data loss or incorrect mappings.
- **Flow Synthesis Correctness:** Confirming that `generate_flow.py` successfully generated valid JSON syntax that Node-RED could import without syntax errors.
- **Topic Verification:** Ensuring that the generated MQTT topics exactly matched the structural hierarchy dictated by the C-header and the runtime prefix arguments.

4.2 System Validation via PLECS Simulation

While verification proves the code executes correctly, validation confirms that the generated SCADA dashboard fulfills its operational purpose. To achieve this safely and reliably before deploying to physical hardware, a digital twin approach was utilized.

The validation environment was established using a **PLECS (Piece-wise Linear Electrical Circuit Simulation)** model. The simulation utilized the `posix_mqttmng_example` implementation to bridge the simulated power electronics hardware with the MQTT broker.

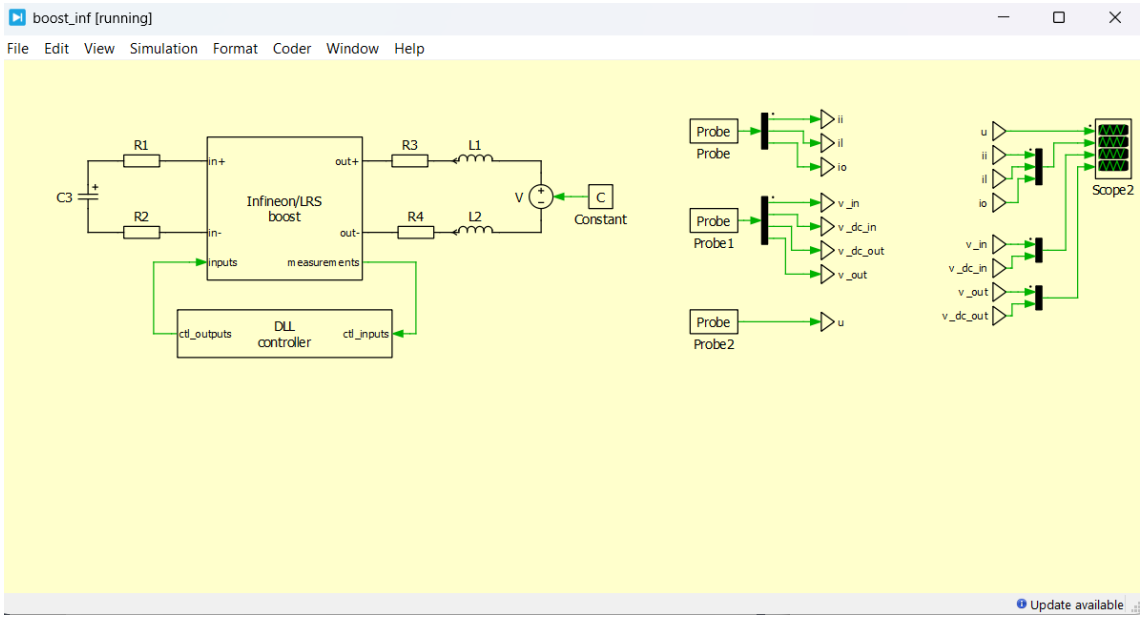

Figure 4.2: PLECS simulation environment (`boost_inf`) acting as the digital twin field device, capturing real-time telemetry such as voltage and current signals via simulated probes.

4.2.1 End-to-End Functional Testing

With the PLECS simulator actively running and transmitting data via MQTT, the dynamically generated Node-RED dashboard was imported and deployed. The following operational criteria were validated:

- **Telemetry Visualization:** The dashboard successfully ingested the binary MQTT payloads, decoded the `Float32` variables via the generated JavaScript function nodes, and accurately plotted the signals on the dynamically assigned chart widgets.
- **Command Execution:** Supervisory commands (e.g., Run, Shutdown) triggered from the UI were successfully published, received by the `posix_mqttmng_example` client, and executed within the PLECS simulation, confirming bidirectional communication.
- **Fault Handling:** Simulated fault conditions injected into the PLECS model correctly updated the bit-shifted fault variables, instantly triggering the active fault indicators on the Node-RED UI.

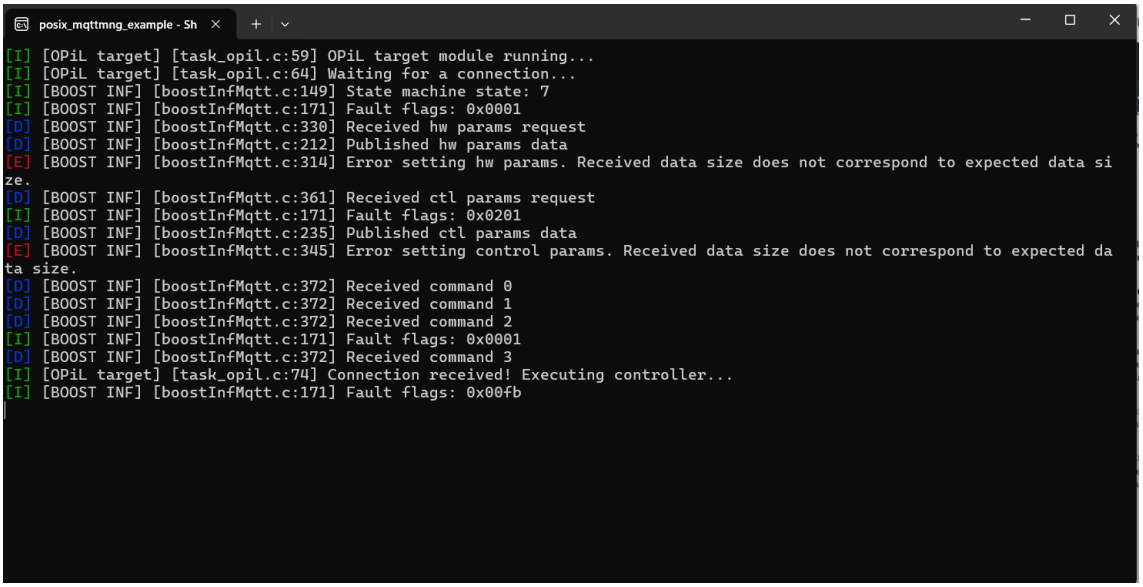

Figure 4.3: Terminal output log from the `posix_mqttmng_example` client, confirming active bidirectional MQTT communication, including telemetry publishing, fault flag updates, and successful command reception.

By utilizing the PLECS simulator alongside the POSIX MQTT client, the automated toolchain was rigorously validated in a closed-loop environment. This proved that the configuration-driven SCADA dashboard not only generates correctly but operates flawlessly as a supervisory interface for the embedded firmware.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

The primary objective of this project was to establish a deterministic, configuration-driven workflow that eliminates the manual effort required to synchronize embedded firmware with SCADA visualization layers. By developing a Python toolchain that extracts operational parameters, MQTT topics, and metadata directly from C-language header files, the firmware was successfully established as the authoritative single source of truth.

The automated generation of deployable Node-RED flows ensures strict structural consistency and type safety. As demonstrated through the PLECS simulation of a Boost converter, the dynamically synthesized dashboard successfully ingested real-time telemetry, decoded binary MQTT payloads, and executed bidirectional supervisory commands without requiring any manual UI configuration. This approach effectively eliminates configuration drift, mitigates human error, and provides a highly scalable framework applicable to diverse power electronic systems, including supercapacitors and active rectifiers.

5.2 Future Work

While the current framework provides a robust and functional foundation for automated SCADA generation, several avenues exist for future enhancement:

- **Protocol Expansion:** The current implementation is strictly bound to MQTT. Extending the generation scripts to support additional industrial communication protocols, such as OPC UA or Modbus TCP, would broaden the toolchain's applicability in legacy industrial environments.
- **Advanced Widget Libraries:** Expanding the `generate_flow.py` script to support a wider array of Node-RED dashboard widgets.The generator could dynamically build more complex UI elements, such as multi-state indicators, dynamic alarm banners.